tieto

360° API Authentication via FK Adgangsstyring Service Accounts

Customer Integration Guide

Contents

Audience **3**

Responsibilities **3**

Authentication Flow Overview **3**

Phase 1: Bootstrap (One-Time) — OAuth 2.0 Device Authorization Grant with PKCE *4*

Phase 2: Runtime (Ongoing) — OAuth 2.0 Refresh Token Grant *5*

Inputs From Tieto **5**

Inputs To Tieto (From Customer) **6**

End-To-End Customer Flow **6**

1. Request integration client setup from Tieto *6*

2. Choose bootstrap approach *6*

3. Implement runtime API client *6*

4. Bootstrap offline token (one-time) *7*

5. Runtime token exchange and API call loop *7*

Runtime Implementation Guidance **7**

1. Exchange offline token for access token *7*

2. Call 360° API *8*

3. Retry and error baseline *8*

Security Rules **8**

Operations (Customer) **8**

Token Revocation **8**

Revoke using CLI *9*

Revoke using HTTP *9*

Troubleshooting **9**

Resources **9**

© Tieto
www.tieto.com

This guide is customer-facing and explains how to authenticate to 360° API using service accounts with FK Adgangsstyring in first phase (no self-service provisioning).

Audience

- Customer integration developers
- Customer integration administrators

Responsibilities

Tieto responsibilities:

- Create the Keycloak bootstrap client for your integration
- Share your `realm-url` and `client-id`
- Provide the bootstrap CLI and sample code for runtime token exchange and API calls if needed

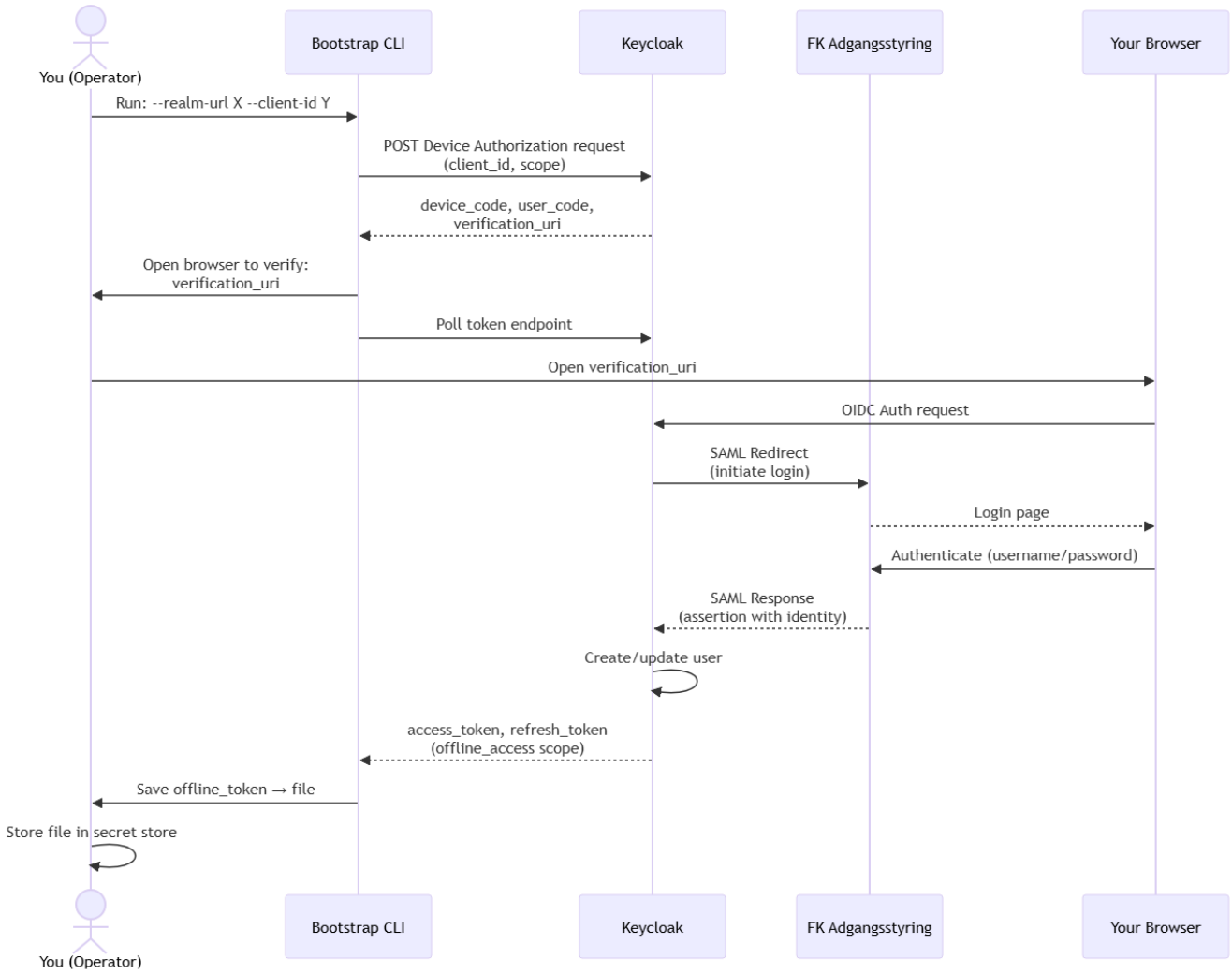
Customer responsibilities:

- Implement runtime API client token exchange (OAuth 2.0 Refresh Token Grant) and API calls. Tieto can provide sample code for this step upon request.
- Run bootstrap once to get an offline token (OAuth 2.0 Device Authorization Grant with PKCE), either with Tieto-provided CLI or with your own implementation.
- Store and protect the offline token in your secret store
- Handle token renewal/recovery at runtime

Authentication Flow Overview

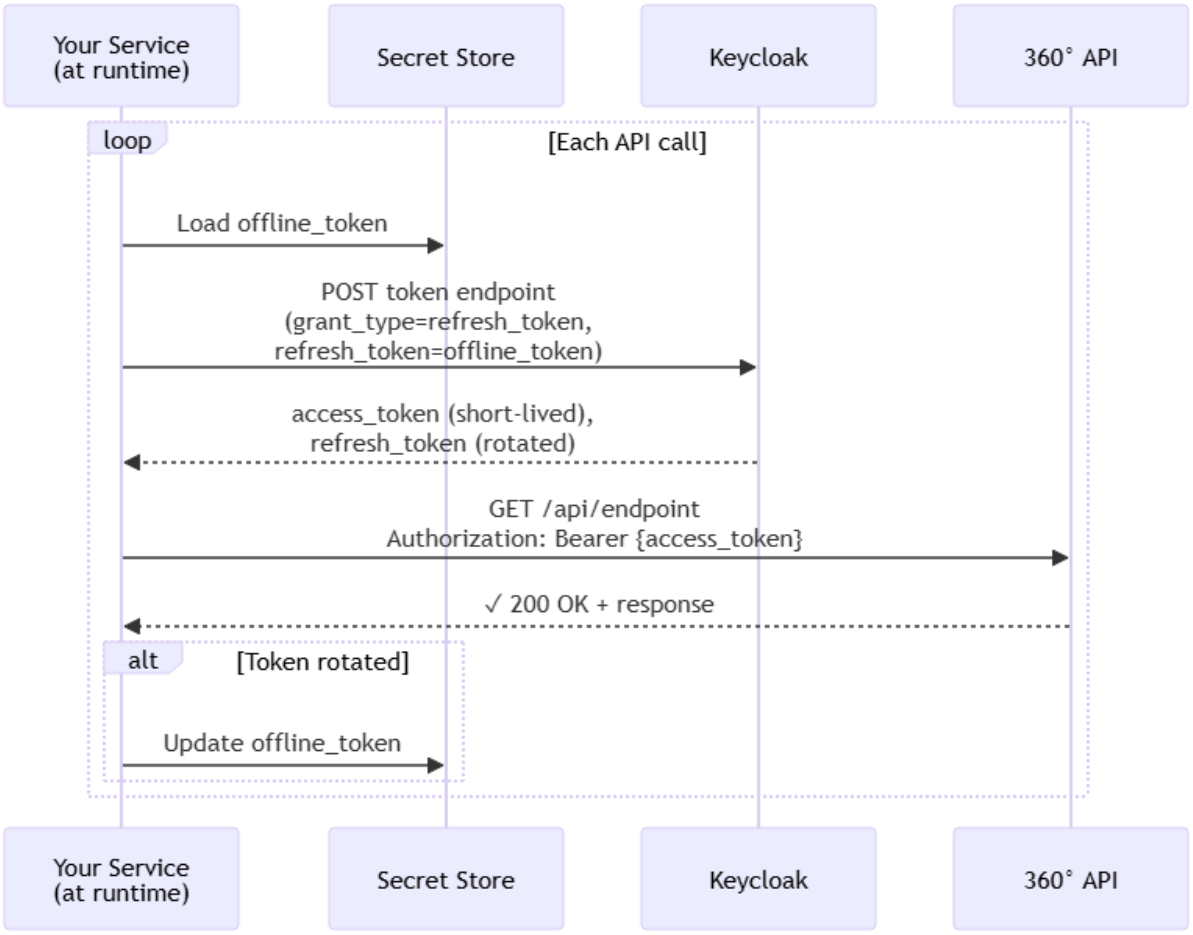
The authentication uses two OAuth 2.0 flows across two phases. Keycloak (on 360° infrastructure) delegates user authentication to FK Adgangsstyring via SAML:

Phase 1: Bootstrap (One-Time) — OAuth 2.0 Device Authorization Grant with PKCE



Output: Offline token (long-lived, refresh token with `offline_access` scope)

Phase 2: Runtime (Ongoing) — OAuth 2.0 Refresh Token Grant



Key Points:

- **Bootstrap**: User authenticates at FK Adgangsstyring (SAML); Keycloak handles OAuth token issuance
- **Runtime**: Service exchanges offline token directly with Keycloak (no FK Adgangsstyring involvement)
- Offline token is a secret (never send to API; never log)
- Access token is short-lived (~10 min); get a fresh one for each API call
- If token exchange fails with `invalid_grant`: offline token is revoked → re-bootstrap
- Keycloak may rotate the refresh token on each exchange; persist it if provided

Inputs From Tieto

Before implementation starts, request the following from your Tieto representative:

- `realm-url` in this format:
  `https://keycloak.360online.com/realms/<customer-realm>`

- `client-id` (recommended: request one client per integration)

Optional

- Bootstrap client (CLI) for your one-time setup. Supported platform-specific executables:
  fk-service-token-cli-linux-x64
  fk-service-token-cli-osx-arm64
  fk-service-token-cli-osx-x64
  fk-service-token-cli-win-x64
- Runtime client sample code (.NET and/or generic HTTP)

Inputs To Tieto (From Customer)

When requesting client setup, provide the following to your Tieto representative:

- Target 360 tenant hostname (for example `customer.public360online.com`)
- Intended client type/purpose
- If multiple clients are expected: a unique client name/identifier per client so each integration can be provisioned separately

If only one client is expected, the default single-client setup is normally sufficient.

End-To-End Customer Flow

1. Request integration client setup from Tieto

Request bootstrap client creation and receive:

- Target tenant confirmation (for example `customer.public360online.com`)
- Realm URL
- Client ID

If multiple clients are needed, include a client list with explicit type/name/identifier for each client.

2. Choose bootstrap approach

Choose one of the following options:

- Option A: Request and download the bootstrap CLI package from Tieto (recommended for fastest onboarding)
- Option B: Implement bootstrap flow yourself using OAuth 2.0 Device Authorization Grant with PKCE

3. Implement runtime API client

Implement your service to:

- Read offline token from secure storage
- Exchange offline token at Keycloak token endpoint (OAuth 2.0 Refresh Token Grant)
- Use short-lived access token for API requests
- Persist updated refresh/offline token if rotation is enabled

4. Bootstrap offline token (one-time)

Use either of the following:

- Option A: Tieto-provided CLI
- Option B: Your own bootstrap implementation (same realm/client, Device Authorization Grant with PKCE)

CLI example:

```
./FkServiceTokenCli --realm-url
"https://keycloak.360online.com/realms/<customer-realm>" --
client-id "<bootstrap-client-id>" --output-mode file --file-
output-format json --token-output-path "./fk-api-offline-
token.local.json" --force
```

Store the output token document in your approved secret storage.

5. Runtime token exchange and API call loop

At runtime, your service should:

1. Load stored offline token
2. Exchange for access token from Keycloak
3. Call 360° API with `Authorization: Bearer <access_token>`
4. Persist latest refresh/offline token if present
5. Repeat for subsequent API calls

Runtime Implementation Guidance

Use this HTTP baseline for all platforms.

If you need full runnable examples (.NET and generic HTTP), request sample package(s) from your Tieto representative.

1. Exchange offline token for access token

Token endpoint format: `https://keycloak.360online.com/realms/<customer-realm>/protocol/openid-connect/token`. You can also discover the token endpoint dynamically from the realm's well-known configuration at `{realm-url}/.well-known/openid-configuration` (recommended for resilience to server changes).

```
POST {token-endpoint} Content-Type: application/x-www-form-urlencoded grant_type=refresh_token&client_id={client-id}&refresh_token={offline-token}
```

Use `access_token` from the response. If `refresh_token` is returned and differs from stored value, update stored token.

2. Call 360° API

```
GET {api-url} Authorization: Bearer {access-token}
```

3. Retry and error baseline

- `401`: exchange token again and retry once
- `429`: honor `Retry-After`; retry with backoff
- `5xx`: retry with exponential backoff
- `invalid_grant` from token endpoint: token unusable, run bootstrap again

Security Rules

- Treat offline token as a secret equivalent to a password.
- Never store token in source code, plain config files, logs, tickets, or screenshots.
- Never send offline token to the API; only send short-lived access token.

Operations (Customer)

- Re-bootstrap when token is revoked, expired, or suspected leaked.
- Re-bootstrap when FK service-user privileges change and must be reflected.
- If token exchange returns `invalid_grant`, treat token as unusable and run bootstrap again.

Token Revocation

Token revocation immediately invalidates the offline token at Keycloak, preventing further token exchanges. Use revocation when:

- Suspected token leak or compromise
- Service decommissioning
- Privilege changes required immediately (cannot wait until next bootstrap)
- Intentional service shutdown

© Tieto

Revoke using CLI

```
./FkServiceTokenCli --realm-url
"https://keycloak.360online.com/realms/<customer-realm>" --
client-id "<bootstrap-client-id>" --revoke --revoke-token-path
"./fk-api-offline-token.local.json"
```

Revoke using HTTP

```
POST {realm-url}/protocol/openid-connect/revoke Content-Type:
application/x-www-form-urlencoded token={offline-
token}&client_id={client-id}&token_type_hint=refresh_token
```

After revocation:

- The offline token becomes invalid immediately
- Any in-flight token exchanges will fail with `invalid_grant`
- Your service must re-bootstrap to obtain a new offline token
- Runtime operations cannot proceed until bootstrap is re-run

Troubleshooting

- `invalid_grant`: token revoked/expired or client mismatch. Re-bootstrap and update stored token.
- No `refresh_token` returned: verify `offline_access` is allowed for the client.
- API 401/403 with valid token: confirm service-user privilege mapping and audience/scope expectations with Tieto.

Resources

Click one of the following links to download the relevant bootstrap CLI package:

fk-service-token-cli-linux-x64

fk-service-token-cli-osx-arm64

fk-service-token-cli-osx-x64

fk-service-token-cli-win-x64

© Tieto
www.tieto.com